

XSS (CROSS-SITE SCRIPTING) DENGAN PYTHON

XSS (Cross-Site Scripting) adalah celah keamanan web di mana penyerang menyisipkan script berbahaya (biasanya JavaScript) ke dalam aplikasi web, lalu dijalankan di browser korban.

Python sendiri **bukan penyebab XSS**, tetapi aplikasi web yang dibuat dengan Python (misalnya pakai Flask atau Django) bisa rentan jika tidak aman dalam menangani input pengguna.

Jenis-Jenis XSS

1. Stored XSS (Persistent)

- ✓ Script disimpan di database

Contoh: komentar berisi `<script>alert('hack')</script>`

2. Reflected XSS

- ✓ Script berasal dari URL/parameter

Contoh:

`http://site.com?q=<script>alert(1)</script>`

3. DOM-Based XSS

Terjadi di sisi frontend (JavaScript)

Contoh XSS dengan Python (Flask)

Contoh Rentan

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def index():
    name = request.args.get("name")
    return f"Halo {name}"
```

Jika user akses:

```
http://localhost:5000/?name=<script>alert('XSS')</script>
```

Script akan dijalankan di browser

Cara Mencegah XSS

1. Gunakan Auto Escaping (Sangat Penting)

Framework seperti Flask & Django sudah mendukung ini.

Aman (Flask + Jinja2)

```
from flask import Flask, render_template_string, request
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def index():
```

```
    name = request.args.get("name")
```

```
    return render_template_string("Halo {{ name }}", name=name)
```

Output:

```
Halo <script>alert('XSS')</script>
```

(Tidak dieksekusi)

2. Validasi & Sanitasi Input

Gunakan library seperti: **bleach**

```
import bleach
```

```
clean = bleach.clean(user_input)
```

3. Gunakan Content Security Policy (CSP)

Header HTTP untuk membatasi script:

```
@app.after_request
```

```
def set_security_headers(response):
```

```
    response.headers['Content-Security-Policy'] = "default-src 'self'"
```

```
    return response
```

4. Hindari |safe di Template

Contoh berbahaya:

```
{{ user_input | safe }}
```

5. Escape Output Secara Manual (Jika Perlu)

```
from markupsafe import escape  
safe_input = escape(user_input)
```

Tips Penting

- Jangan pernah percaya input user
- Gunakan template engine (Jinja2, Django Template)
- Hindari langsung return string HTML
- Selalu lakukan encoding output

Kesimpulan

XSS terjadi karena:

Input user langsung ditampilkan tanpa difilter

Solusi utama:

- ✓ Escape output
- ✓ Gunakan template engine
- ✓ Sanitasi input

LAB LANJUTAN: XSS DAN FILE UPLOAD ATTACK (HACK VS SECURE)

Tujuan

Siswa mampu:

- Memahami dan mengeksploitasi XSS
- Memahami File Upload Vulnerability
- Mengamankan aplikasi dengan best practice

1. Setup Project

Install:

```
pip install flask
```

Struktur folder:

```
project/  
|— app.py  
|— uploads/
```

2. LAB XSS (Vulnerable)

Kode Rentan

```
from flask import Flask, request, render_template_string  
  
app = Flask(__name__)  
  
@app.route("/", methods=["GET", "POST"])  
def index():  
    comment = ""  
    if request.method == "POST":  
        comment = request.form["comment"]  
  
    return render_template_string("""  
        <h2>Komentar</h2>
```

```
<form method="POST">
  <input name="comment">
  <button type="submit">Kirim</button>
</form>
<p>Komentar: "" + comment + ""</p>
""")

app.run(debug=True)
```

Serangan XSS

Input:

```
<script>alert('XSS')</script>
```

Hasil:

Script langsung dijalankan di browser

Perbaikan XSS

Ganti dengan template aman:

```
return render_template_string("""
  <h2>Komentar</h2>
  <form method="POST">
    <input name="comment">
    <button type="submit">Kirim</button>
  </form>
  <p>Komentar: {{ comment }}</p>
""", comment=comment)
```

Output di-escape otomatis → tidak dieksekusi

3. LAB File Upload (Vulnerable)

Kode Rentan

```
from flask import Flask, request
import os
```

```

app = Flask(__name__)

UPLOAD_FOLDER = "uploads"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

@app.route("/upload", methods=["GET", "POST"])
def upload():
    if request.method == "POST":
        file = request.files["file"]
        file.save(os.path.join(app.config["UPLOAD_FOLDER"], file.filename))
        return "Upload berhasil"

    return """
    <form method="POST" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
    </form>
    """

```

Serangan File Upload

Upload file: evil.html

Isi:

```
<script>alert('Hacked via upload')</script>
```

Akses:

```
http://127.0.0.1:5000/uploads/evil.html
```

Terjadi XSS dari file upload

4. Perbaiki File Upload

Versi Aman

```

from flask import Flask, request, abort
from werkzeug.utils import secure_filename

```

```
import os, uuid

app = Flask(__name__)

UPLOAD_FOLDER = "uploads"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg"}

def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in
ALLOWED_EXTENSIONS

@app.route("/upload", methods=["POST"])
def upload():
    if "file" not in request.files:
        abort(400)

    file = request.files["file"]

    if file.filename == "":
        abort(400)

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        filename = str(uuid.uuid4()) + "_" + filename
        file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))
        return "Upload aman"

    return "File tidak diizinkan"
```

Perbandingan

Fitur	Vulnerable	Secure
XSS	Bisa inject script	Auto escape
Upload file	Semua file bebas	Filter ekstensi
Nama file	Tidak aman	secure_filename
Risiko	Tinggi	Rendah

Diskusi (Untuk Siswa)

1. Kenapa `<script>` bisa dieksekusi?
2. Apa perbedaan render string vs template engine?
3. Kenapa upload file bisa jadi XSS?
4. Apa risiko jika upload .py atau .php?

Challenge (Level Lanjut)

- Upload file dengan bypass extension (.jpg.php)
- Kombinasikan XSS + SQL Injection
- Tambahkan validasi MIME type
- Integrasi dengan login JWT

Kesimpulan

- XSS terjadi karena output tidak di-escape
- File upload bisa jadi pintu masuk serangan
- Validasi + sanitasi = wajib